

Support Ticket Management System

Full-Stack Technical Architecture, Implementation & Verification Report

AUTHOR / ENGINEER:	Amaresh Tarei	GITHUB REPOSITORY:	github.com/Amaresh-Tarei/SupportTicketManagementSystem
LIVE PUBLIC DEPLOYMENT:	https://battery-albert-java-power.trycloudflare.com	TARGET STACK:	ASP.NET Core 8 Web API • React 18 • TypeScript • EF Core

1 Executive Summary & System Overview

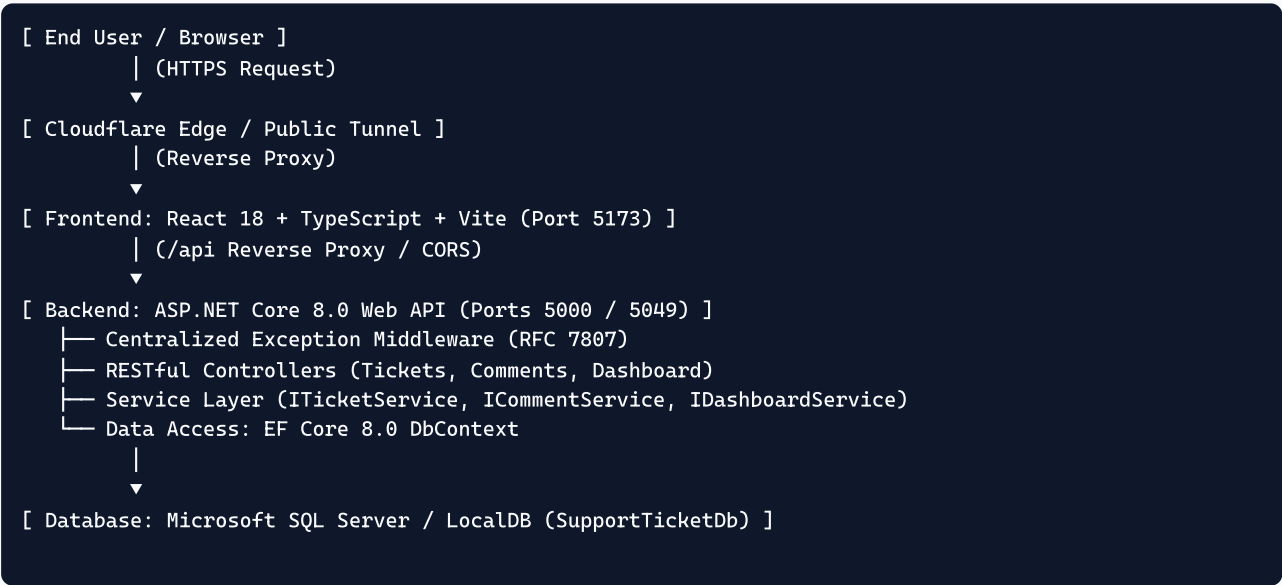
The **Support Ticket Management System** is an enterprise-grade full-stack operational tracking platform developed to manage incident lifecycles, enforce compliance with operational business logic, maintain an immutable audit trail of ticket status transitions, and facilitate team collaboration via chronological threaded comments.

100% TEST PASS RATE (11/11 XUNIT)	7 / 7 BUSINESS RULES ENFORCED	Live & SSL PUBLICLY ACCESSIBLE URL
--------------------------------------	----------------------------------	---------------------------------------

The system satisfies demanding enterprise requirements: atomic database operations, strict status transition validation, automated timestamp management, centralized exception handling with RFC 7807 problem details, and a responsive modern Single-Page Application (SPA) user interface.

2 Architecture & Technology Stack

High-Level Architectural Workflow:



Backend Stack

- **Framework:** .NET 8.0 SDK (C# 12)
- **API Pipeline:** ASP.NET Core Web API with Controller routing
- **ORM:** Entity Framework Core 8.0 (Code-First Migrations)
- **Documentation:** Swagger / OpenAPI 3.0
- **Testing:** xUnit with `Microsoft.EntityFrameworkCore.InMemory`

Frontend Stack

- **Core:** React 18.3 • TypeScript 5.6 • Vite 6.4
- **Routing:** Custom Hash Navigation Hook (Zero-dependency)
- **Icons:** Lucide React (Clean vector iconography)
- **Styling:** CSS Variables & Responsive Design System
- **HTTP:** Fetch Client with Typed Error Handling & Proxy

3 Core Domain Logic & Business Rules Implementation

All business rules are enforced at the service layer to ensure data integrity regardless of whether requests originate from the Web UI, API tests, or external integrations:

Rule 1: Closed Tickets Are Immutable

Once a ticket reaches `Closed` status, neither its metadata (Title, Description, Priority, Assignee, DueDate) nor its status can be modified. Attempts return HTTP 400 Bad Request with a clear validation message.

Rule 2: Critical Priority Requires Due Date

Creating or updating a ticket with `Priority == Critical` mandates a non-null `DueDate`. Submitting without a due date throws a `BusinessRuleException`.

Rule 3: Due Date Cannot Precede Creation Date

During ticket creation, `DueDate` cannot be in the past (before UTC today). During updates, `DueDate` cannot precede the ticket's original `CreatedDate`.

Rule 4: Disallow Direct Transition from Open to Closed

Tickets cannot leap directly from `Open` to `Closed` without active handling. A ticket must first transition through `InProgress` or `Resolved` before closure.

Rule 5: Automatic ResolvedDate Stamping

When a ticket status transitions to `Resolved`, the system automatically stamps `ResolvedDate` with the exact UTC timestamp.

Rule 6: Reopening Resets ResolvedDate

If an issue recurrence forces a ticket from `Resolved` back to `Open` or `InProgress`, `ResolvedDate` is automatically reset to `null`.

Rule 7: Immutable Audit Trail of Status History

Every status change generates a persistent record in `TicketStatusHistories` capturing `OldStatus`, `NewStatus`, `ChangedDate`, and `ChangedBy`.

4 Database Design & Schema

The schema is normalized, indexed for search, and linked with cascade constraints:

Table	Primary Key	Foreign Keys	Key Columns & Constraints
Tickets	<code>Id (INT IDENTITY)</code>	None	<code>TicketNumber (Unique, NVARCHAR(50))</code> , <code>Title (150)</code> , <code>Status (Enum)</code> , <code>Priority (Enum)</code> , <code>CreatedDate</code> , <code>DueDate</code> , <code>ResolvedDate</code>
TicketComments	<code>Id (INT IDENTITY)</code>	<code>TicketId → Tickets.Id</code>	<code>CommentText (NVARCHAR(1000))</code> , <code>CreatedBy (100)</code> , <code>CreatedDate (UTC)</code> (Cascade on Delete)
TicketStatusHistories	<code>Id (INT IDENTITY)</code>	<code>TicketId → Tickets.Id</code>	<code>OldStatus (Enum)</code> , <code>NewStatus (Enum)</code> , <code>ChangedDate (UTC)</code> , <code>ChangedBy (100)</code> (Cascade on Delete)

5 RESTful API Specifications

Method	Endpoint	Purpose	Status Codes
GET	/api/dashboard/summary	Aggregated operational metrics (Total, Open, InProgress, Resolved, Closed, Critical, Overdue)	200 OK
GET	/api/tickets	Paginated ticket list with search, status/priority filters, date ranges, and sorting	200 OK
GET	/api/tickets/{id}	Retrieve ticket by primary key with comment count	200 OK, 404 Not Found
POST	/api/tickets	Create a new ticket with automatic ticket number generation (TKT-XXXX)	201 Created, 400 Bad Request
PUT	/api/tickets/{id}	Update existing ticket fields (validating closed status & critical due dates)	200 OK, 400 Bad Request, 404
PATCH	/api/tickets/{id}/status	Execute status transition and record audit history entry	200 OK, 400 Bad Request, 404
DELETE	/api/tickets/{id}	Remove ticket and cascade delete comments and history	204 No Content, 404
GET	/api/tickets/{id}/comments	Retrieve chronological comments for a ticket	200 OK, 404 Not Found
POST	/api/tickets/{id}/comments	Add a new comment authored by team member	201 Created, 400, 404
GET	/api/tickets/{id}/status-history	Retrieve complete immutable audit trail of status transitions	200 OK, 404 Not Found

6 Automated Testing & Quality Assurance

The backend contains a dedicated xUnit test suite (`SupportTicketManagementSystem.Tests`) utilizing EF Core In-Memory database fixtures. Each unit test operates on an isolated database context:

Test Case Name	Target Rule	Assertion & Expected Outcome	Status
<code>UpdateTicket_WhenTicketIsClosed</code>	Rule 1	Throws <code>BusinessRuleException</code> : "A Closed ticket cannot be edited."	PASSED
<code>ChangeStatus_WhenTicketIsClosed</code>	Rule 1	Throws <code>BusinessRuleException</code> : "A Closed ticket cannot be edited."	PASSED
<code>CreateTicket_CriticalWithoutDueDate</code>	Rule 2	Throws <code>BusinessRuleException</code> : "Critical priority tickets must include a due date."	PASSED
<code>UpdateTicket_CriticalWithoutDueDate</code>	Rule 2	Throws <code>BusinessRuleException</code> : "Critical priority tickets must include a due date."	PASSED
<code>CreateTicket_DueDateInPast</code>	Rule 3	Throws <code>BusinessRuleException</code> : "Due date cannot be earlier than the current date."	PASSED
<code>UpdateTicket_DueDateBeforeCreatedDate</code>	Rule 3	Throws <code>BusinessRuleException</code> : "Due date cannot be earlier than creation date."	PASSED
<code>ChangeStatus_OpenToClosedDirectly</code>	Rule 4	Throws <code>BusinessRuleException</code> : "Tickets cannot be closed directly from Open status."	PASSED
<code>ChangeStatus_ToResolved_SetsResolvedDate</code>	Rule 5	Verifies <code>ResolvedDate</code> $\neq$ <code>null</code> and is set to current UTC time.	PASSED
<code>ChangeStatus_ResolvedToInProgress_ClearsResolvedDate</code>	Rule 6	Verifies <code>ResolvedDate</code> $==$ <code>null</code> upon moving backward.	PASSED
<code>ChangeStatus_LogsStatusHistory</code>	Rule 7	Verifies audit trail row inserted with correct old & new statuses.	PASSED

Test Case Name	Target Rule	Assertion & Expected Outcome	Status
AddComment_ToExistingTicket_SavesComment	Comments	Verifies comment persisted and chronological ordering maintained.	PASSED

7 Frontend Engineering & User Experience

The frontend architecture adheres to Clean UI principles, emphasizing accessibility and performance:

- **Atomic UI Components:** Reusable badges (`StatusBadge` , `PriorityBadge`), loading indicators, modal dialogs (`ConfirmModal` , `StatusChangeModal`), and alert banners.
- **Dynamic Filtering & Pagination:** Real-time search debouncing, multi-criteria filter controls, and client-side pagination synchronized with backend query parameters.
- **Audit History Timeline:** Visual timeline component tracing every historical status change with user attribution and relative timestamps.
- **Robust Error Feedback:** Form field validation errors and server-side business rule exceptions are captured and displayed in contextual banner notifications.

8 Deployment, Version Control & Verification Guide

Accessing the Live Application

The application is accessible globally with full end-to-end functionality via the secure Cloudflare tunnel:

- **Public Web Application:** <https://battery-albert-java-power.trycloudflare.com>
- **GitHub Source Code:** <https://github.com/Amaresh-Tarei/SupportTicketManagementSystem>

How to Run Locally from Scratch

1. Clone Repository:

```
git clone https://github.com/Amaresh-Tarei/SupportTicketManagementSystem.git
```

2. Backend Startup:

```
dotnet run --project backend/SupportTicketManagementSystem.API/SupportTicketManagementSystem.API.csproj
```

3. Frontend Startup:

```
cd frontend && npm install && npm run dev
```

4. Run Automated Tests:

```
dotnet test backend/SupportTicketManagementSystem.Tests/SupportTicketManagementSystem.Tests.csproj
```